第4回：断面配置・押し出し・メッシュ生成のC#実装例

詳細解説 と 宿題の問題・解答 の固定例A〜Dで、断面の配置から8頂点・12三角形の生成までを追う。`double` の成分式で配置・内積・外積・行列変換を計算し、`System.Numerics` の `float` とも照合する。

ファイルと読む順序

ファイル	数学的な役割
Program.cs	固定入力、独立な期待値、7題材、出力と実行時チェック
ModelingMath.cs	doubleの2D／3Dタプル、フレーム、成分式、列行列、行方式への変換、符号付き断面積
MeshBuilder.cs	頂点・三角形・面の計算結果の型、凸四角形の押し出し、辺の接続と符号付き体積

`Program.Main` → 固定入力 → 配置式 → 頂点配列 → 三角形配列 → 法線と面積 → 接続と体積の順に読む。日本語XMLコメントには入力・結果・制約を記載している。

題材・固定入力・期待結果

題材	解説／宿題との対応	主な入力・期待値	チェック数
1. LocalPlacement	②／基本1-1,2	A：C=(10,20,30)、N=+Y、B=+Z、T=+X、局所(2,−1) → (10,22,29)。直交単位基底、API照合	8
2. HomogeneousMatrix	②／基本1-3,4	N,B,T,Cの列方式。w=0・同次成分1 → (10,22,29,1)。行方式へ転置してVector3/4と一致	8
3. ExtrusionVertices	③／基本2-1,4	B：局所(−1,−0.5)→(1,−0.5)→(1,0.5)→(−1,0.5)。始端X=0、終端X=5の8点	9
4. FacesAndNormals	④／基本3-1,2	12三角形・36インデックス。全12枚の順序、3点、外積、単位法線、面積と外向き	62
5. ClosureAndVolume	⑤／基本2-2,3・基本3-3,4	断面積2×長さ5=10、辺18本の逆向き共有、独立な符号付き体積10。表面積34は別の値	7
6. StationFrames	⑥・⑦／発展	C：始点(0,1,0)、終点(4,4,0)、姿勢固定なら(5,5,0)、角の片側配置。D：回転点(0,2,1)	11
7. InvalidInputsAndPrecision	⑧／基本3・発展の注意	退化・逆順・非有限値・桁あふれ・開口を拒否。全反転の体積−10、floatで微小な差が消える	48

出力例（抜粋）：

```
  [OK] A: C+2N-B: 10, 22, 29
  [OK] A: 転置した行方式Vector4: 10, 22, 29, 1
  [OK] B: 頂点0: 0, -1, -0.5
  [OK] B: 頂点7: 5, -1, 0.5
  [OK] B: 三角形数: 12
  [OK] B: インデックス数: 36
  [OK] B: 三角形0の順序: 0, 1, 5
    3点: (0,-1,-0.5) / (0,1,-0.5) / (5,1,-0.5)
  [OK] B: 三角形0の外積: 0, 0, -10
  [OK] B: 三角形0の単位法線: 0, 0, -1
...
  [OK] B: 断面積×法線方向の長さ: 10
  [OK] B: 符号付き体積: 10
  [OK] C: 終点の局所(1,0): 4, 4, 0
...
すべての 153 チェックに成功しました。
```

チェックは未丸めの値を比較する。表示は通常G9、失敗時はG17とする。`-0` は浮動小数点の符号付きゼロで、期待する0と数値的に等しい。

座標・頂点・面の規約

`Place` はフレームを検証し、`C+uN+vB` で断面点を配置する。`CreateVertices` は `vertices[i]` に始端、`vertices[i + 4]` に終端を置くため、固定例BではX=0とX=5に同じY・Zを持つ8点ができる。

- 右手系XYZ。V2は局所(u,v)、V3は世界(X,Y,Z)のdoubleタプル。点と方向を同じ型で表すため、文脈とコメントで区別する。
- N・Bは断面内の直交単位基底、T=N×B。輪郭は **+T側から断面の原点へ見る反時計回り** とする。u右・v上の局所座標に対応する。
- 始端i=0〜3、終端i+4=4〜7。`Vertices` は共有座標、`Triangles` は0始まりの順序付き3番号。三角形の各点は `Vertices[face.A]` などで取得する。
- 輪郭の次をj=(i+1) mod 4として、側面は(i,j,j+4)と(i,j+4,i+4)。最初の側面は(0,1,5)と(0,5,4)になる。内部を左に見る輪郭の辺eに対し、e×Tが右側の外向きになる。
- 始端キャップは(0,2,1)・(0,3,2)で−T、終端は(4,5,6)・(4,6,7)で+T。対応頂点の保存順は変えず、キャップの順序を変える。
- `AnalyzeTriangle` は2辺の差から外積 `(b-a)×(c-a)` を求め、その長さを三角形面積の2倍として退化検査し、検査後に単位化する。底面の外積(0,0,−10)と単位法線(0,0,−1)を混同しない。

列方式のdouble配列は基底N,B,Tを列に、Cを第4列に置く。断面点の局所w=0と同次成分1は別物である。`System.Numerics` は行ベクトルvMの規約なので、`NumericsRowMatrix` は **Matrix4x4.Transposeで転置してから** `Vector3.Transform`／`Vector4.Transform` に渡す。公式の行方式の説明 と照合できる。

APIの入力制約と結果の意味

`CreateVertices`／`ExtrudeConvexQuad` は、**有限な4点、重複なし、反時計回りの厳密に凸な輪郭、右手系の直交単位基底、正の押し出し長**を要求する。非凸・自己交差・穴あり・一般多角形の三角形分割は扱わない。今回の8頂点・12三角形を任意のCAD形状へ一般化しない。

輪郭は先頭点を末尾に重複保存しない。配列だけでは最後の辺を閉じる意図が分からないため、`closed` を明示する。trueで末尾→先頭を接続し、falseは拒否する。始点を重複保存する形式の自動変換はしない。

入力制約違反、非有限な中間値、下限以下の形状には `ArgumentException` を投げ、偽の原点・法線を返さない。`CreateVertices` は配置段階、`ExtrudeConvexQuad` はさらに生成した全三角形を検査する段階である。大きな世界座標で異なる局所点が同じ座標へ丸められる場合も、後者の退化検査で拒否する。一般の丸め誤差をすべて検出する保証はない。

`HasPairedOppositeEdges` は各無向辺のCount=2と向きのBalance=0を検査する。片方のキャップが欠けた場合や1枚だけの反転は検出するが、全反転は通る。**辺の対だけで自己交差・頂点周りの非多様体接続・外向き・有効ソリッドを保証しない。** 三角形の幾何も別に検査する。

`SignedVolume` は辺の対と全三角形の非退化を検査してから、基準点oに対する符号付き四面体 `(a-o)・((b-o)×(c-o))/6` を足す。oは最初の頂点とし、大きな共通平行移動による相殺を減らす。本文の世界原点の式と、閉じた境界では同じ体積になる。教材では単純な凸直柱の検算に限り、まず断面積×法線方向の長さで10と確認する。各寄与の絶対値を足さず、全体を反転したら−10になることも確認する。

メッシュは有限個の面による境界表現で、この直柱の平面は正確に表せるが、曲面は通常近似する。CADのB-repソリッドを生成・検証するものではない。`StationFrames` では経路の終点でNが−Xへ変わることと、角の左右のフレームで配置点が異なることを確認する。経路の2フレームの配置と回転点は点の検算に限り、コーナー接続・輸送フレーム・汎用スイープ／回転体メッシュの生成は含めない。

数値精度としきい値

条件	教材の設定・尺度
数値の期待値比較	doubleは絶対誤差1e-12、floatのAPIは1e-5。実値・期待値の両方の有限性も検査
押し出し長・局所点間距離	MinimumLength=1e-12以下を拒否。長さ単位
角の2D外積・三角形の2倍面積	MinimumArea2=1e-12以下を拒否。長さの2乗単位
フレーム	内積・長さの2乗と1の差・N×B−Tの長さにFrameTolerance=1e-12。無次元

これらは固定単位・小さな入力向けの設定で、厳密な幾何判定ではない。長さ、面積、無次元の角度条件へ同じしきい値を無条件に流用しない。単位や尺度を変える際は設定も見直す。桁あふれ・アンダーフロー、座標差の桁落ち、境界付近の誤差への汎用的な対策は範囲外である。

`System.Numerics` の成分は Single（float） で、doubleから渡すと丸められる。題材7の `1+1e-10` の差はdoubleに残るが、floatでは消える。doubleに戻しても復元できない。許容誤差内でも元の情報が完全に保存されたとは限らない。

検証範囲

7題材・153件の自動チェックで、配置・頂点・三角形の順序・法線・面積・辺の共有・符号付き体積を固定値と照合する。全12三角形の幾何、辺の逆向き共有、体積10を別々に検査し、無効入力の拒否と全反転時の体積−10も確認する。固定例の検算であり、有効なCADソリッドや汎用メッシュの検証を保証しない。

資料の元データ（GitHubリポジトリ）： https://github.com/mkunori/geometry-study-group